

maquette-gltf

Render glTF 2.0 assets in Typst

github.com/bernsteining/maquette · bernsteining.github.io/maquette



Version 0.1.0 · August 26, 2026

CONTENTS

Contents

Introduction	3
Quickstart	4
Camera	5
Cartesian	5
Spherical	5
Auto-camera	5
Backgrounds	7
Image-Based Lighting	8
Procedural IBL	8
HDR environment	8
IBL rotation	8
Shadows	9
Ground plane	10
Post-processing	11
Tone mapping	11
Animations	12
Material variants	13
Diffuse transmission	13
Compression + quantization	14
EXT_meshopt_compression	14
KHR_draco_mesh_compression	14
KHR_mesh_quantization	14
Texture formats	15
EXT_texture_webp	15
Unsupported formats	15
Metadata	16
What's supported vs. not	17

Introduction

maquette-gltf is a Typst plugin that renders glTF 2.0 assets — `.glb` or `.gltf` — directly into your documents. It sits alongside `maquette` (STL/OBJ/PLY) and `maquette-scad` (OpenSCAD → mesh) in the same workspace, sharing render primitives via `maquette-core`.

The rendering is a software PBR pipeline: Cook-Torrance GGX + Karis split-sum IBL + shadow maps with PCF/PCSS + WBOIT for translucency, running entirely in Typst’s wasm sandbox. No GPU, no network, no external tooling. Same input → same PDF, every time.

Because glTF assets carry authored materials + cameras + lights + animations, most of the value is in exposing those to the shader knobs without asking you to re-author anything. Drop a `.glb` in, pick an environment map, get a rendered frame.

Quickstart

Two input shapes, both handled transparently.

.glb or fully-embedded .gltf — pass the bytes directly. This is the common case (Damaged Helmet, boombox, most Sketchfab downloads).

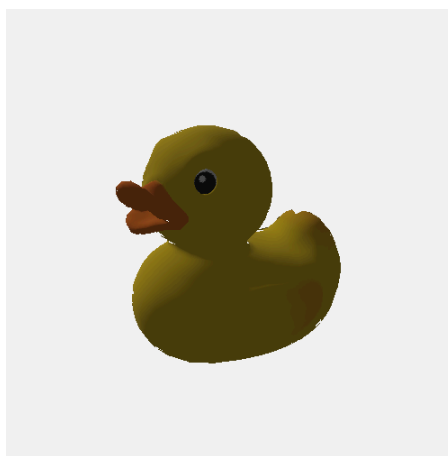
```
#import "@preview/maquette-gltf:0.1.0": render-gltf

#render-gltf(read("helmet.glb", encoding: none))
```



Split .gltf — the .gltf JSON references external .bin + textures by relative URI. Pass the **path string** plus an inline `read: lambda`; the wrapper walks the JSON, discovers every URI, reads each one through your lambda, packs them into a sidecar bundle for the plugin. You write zero filenames.

```
#render-gltf("Duck.gltf", read: p => read(p, encoding: none))
```



The `read: lambda` has to be an **inline lambda**, not a bare read reference. Typst resolves `read()` paths against the source file the call is textually in — a bare reference stays bound to the package's own path context. Wrapping in `p => read(p, ...)` gives the wrapper a filesystem handle scoped to your `.typ`.

Camera

Two ways to frame a shot: give an explicit Cartesian position, or (rare for glTF) drive it spherically. When a .glb ships an authored camera and you don't override framing, that authored camera wins.

Cartesian

```
#render-gltf(helmet, read: R,  
  camera: (2.5, 1.5, 2.5),  
  center: (0, 0, 0),  
  up: (0, 1, 0),  
  fov: 40,  
)
```



Spherical

```
#render-gltf(helmet, read: R,  
  azimuth: 30, elevation: 15,  
  up: (0, 1, 0),  
  fov: 40,  
)
```



Auto-camera

Set `camera_auto_use`: `false` to force the orbit fallback even when the asset ships a camera. Default is `true` (viewer convention).

```
#render-gltf(helmet, read: R,  
  camera_auto_use: false,  
  camera: (0, 0, 3),  
  up: (0, 1, 0),  
  fov: 60,  
)
```



Backgrounds

Solid color, "none" for transparent, or #hex.

```
#render-gltf(helmet, read: R,  
  camera: (2.5, 1.5, 2.5), up: (0, 1, 0), fov: 40,  
  background: "#334",  
)
```



```
#render-gltf(helmet, read: R,  
  camera: (2.5, 1.5, 2.5), up: (0, 1, 0), fov: 40,  
  background: "none",  
)
```



Image-Based Lighting

The dominant visual lever for PBR: an environment map lights the whole scene. Enabling ibl without an hdr bytes payload falls back to a procedural hemispheric sky.

Procedural IBL

```
#render-gltf(helmet, read: R,  
  camera: (2.5, 1.5, 2.5), up: (0, 1, 0), fov: 40,  
  background: "#181820",  
  ibl: (intensity: 1.2),  
)
```



HDR environment

Pass Radiance .hdr bytes as ibl.hdr. The prefilter builds a diffuse irradiance map + a specular mip chain up-front (once per HDR, cached).

```
#render-gltf(helmet, read: R,  
  ibl: (intensity: 1.0, hdr: read("studio.hdr", encoding: none)),  
)
```

IBL rotation

The rotation field spins the environment around the up axis (radians). Useful for aligning studio HDRs to your camera composition.

```
#render-gltf(helmet, read: R,  
  camera: (2.5, 1.5, 2.5), up: (0, 1, 0), fov: 40,  
  background: "#181820",  
  ibl: (intensity: 1.2, rotation: 1.2),  
)
```



Shadows

Shadow maps per punctual light with PCF or PCSS. shadows on its own applies safe defaults; the dict form exposes every knob.

```
#render-gltf(helmet, read: R,  
  camera: (2.5, 1.5, 2.5), up: (0, 1, 0), fov: 40,  
  background: "#181820",  
  ibl: (intensity: 1.0),  
  shadows: (resolution: 1024, softness: 1, bias: 0.001),  
  ground: (color: "#282838", size_scale: 3.0, roughness: 0.9),  
)
```



Bumping pcss_light_size from 0 to a small value switches to PCSS soft shadows — occluder search + penumbra estimate per pixel, larger light_size gives softer edges.

Ground plane

An axis-aligned rectangle beneath the model that catches shadows + reflects the environment. Materialized as a matte dielectric with configurable colour + roughness.

```
#render-gltf(helmet, read: R,  
  camera: (2.5, 1.5, 2.5), up: (0, 1, 0), fov: 40,  
  background: "#181820",  
  ibl: (intensity: 1.2),  
  ground: (color: "#282838", size_scale: 3.0, roughness: 0.9),  
)
```



Post-processing

SSAO (screen-space ambient occlusion), FXAA, and SSAA (2×/4×). Ambient-occlusion has the biggest visual lift for cheap.

```
#render-gltf(helmet, read: R,  
  camera: (2.5, 1.5, 2.5), up: (0, 1, 0), fov: 40,  
  background: "#181820",  
  ibl: (intensity: 1.2),  
  ssao: (samples: 16, radius: 0.4, strength: 1.0),  
)
```



antialias: 0 off, 1 FXAA, 2 SSAA×2, 4 SSAA×4. FXAA is cheap; SSAA quadruples render cost per level.

Tone mapping

HDR → LDR mapping for physically-motivated exposure. "aces" matches film response; "reinhard" is softer.

```
#render-gltf(helmet, read: R,  
  camera: (2.5, 1.5, 2.5), up: (0, 1, 0), fov: 40,  
  background: "#181820",  
  ibl: (intensity: 1.5),  
  tone_mapping: (method: "aces", exposure: 1.2),  
)
```



Animations

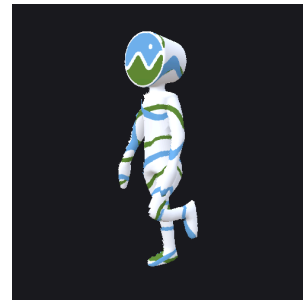
Assets with animation channels get replayed at time seconds. `get-gltf-info` returns `max_animation_time` so you can build a scrub slider bounded to the actual clip length.

```
#render-gltf(fox,
  camera: (120, 90, 180), center: (0, 40, 0), up: (0, 1, 0),
  fov: 40,
  background: "#1a1a22",
  ibl: (intensity: 1.3),
  time: 0.5,
  width: 45%,
)
```



When the asset ships multiple clips (idle / walk / run / ...), pick one with `animation_index`. None (the default) plays every clip stacked — last-write-wins per node channel — which is almost never what you want for multi-clip assets.

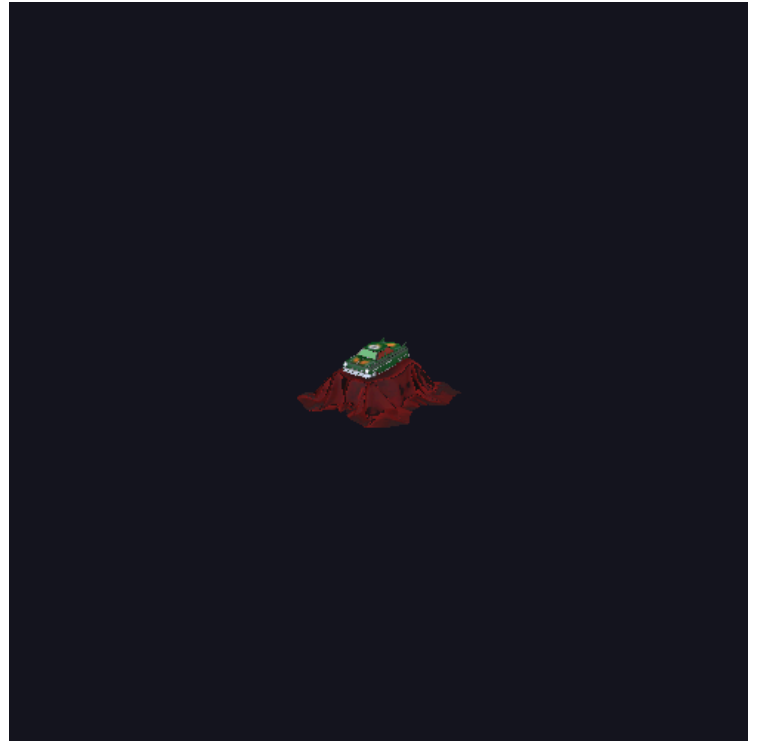
```
#render-gltf(cesiumman,
  camera: (2.5, 1, 2.5), center: (0, 0.9, 0), up: (0, 1, 0),
  fov: 30,
  background: "#1a1a22",
  ibl: (intensity: 1.3),
  animation_index: 0,
  time: 0.3,
  width: 40%,
)
```



Material variants

KHR_materials_variants lets an asset ship multiple material sets. Pick one with `material_variant` (0-indexed).

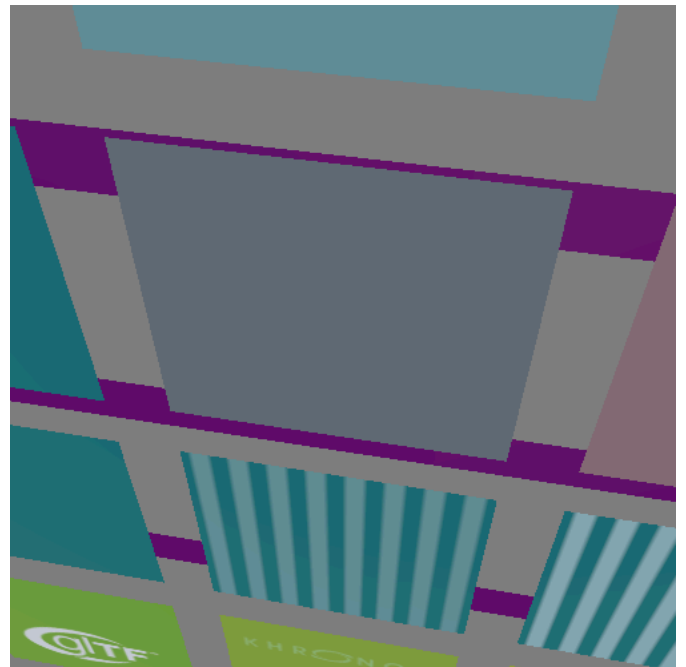
```
#render- gltf(toycar,  
  camera: (0.4, 0.3, 0.55), center: (0, 0.05, 0), up: (0, 1, 0),  
  fov: 30,  
  background: "#181820",  
  ibl: (intensity: 1.4),  
  material_variant: 0,  
)
```



Diffuse transmission

KHR_materials_diffuse_transmission — matte back-lit lambertian. Common on cloth, leaves, paper. The example asset is a factor $\times$ color grid; the bottom rows show the green DT color activating as the factor increases.

```
#render- gltf(dt-test,  
  camera: (1.5, 0.5, 1.5), center: (0, 0, 0), up: (0, 1, 0),  
  fov: 40,  
  background: "#222",  
  ibl: (intensity: 1.0),  
  width: 90%,  
)
```



Compression + quantization

The plugin decodes three common wire formats up-front, so downstream traversal never has to care.

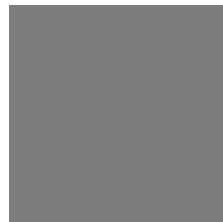
EXT_meshopt_compression

Emitted by gltfpack (Meshopt's own tool). Interleaved buffer decompression, transparent.

KHR_draco_mesh_compression

Google Draco. Decoded via draco-oxide (pure Rust). Up to 20× geometry compression on typical assets, no size penalty at render time.

```
#render-gltf(box-draco, read: R,  
  camera: (2, 1, 2), center: (0, 0, 0), up: (0, 1, 0), fov: 40,  
  background: "#333",  
  width: 30%,  
)
```



KHR_mesh_quantization

gltfpack also emits quantized POSITION/NORMAL/TANGENT (i8/u8/i16/u16) paired with KHR_texture_transform for UV dequantization. Renders pixel-identically to the non-quantized asset — verified on the Duck below.

```
#render-gltf(duck-quant, read: R,  
  camera: (2.5, 1.5, 2.5), center: (0, 0, 0), up: (0, 1, 0),  
  fov: 40,  
  width: 40%,  
)
```



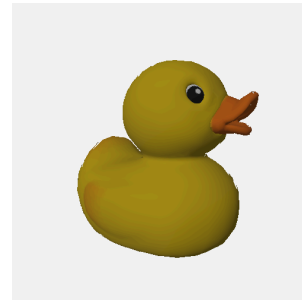
Texture formats

Beyond the base PNG + JPEG:

EXT_texture_webp

image-webp (pure Rust) handles both VP8 and VP8L variants. Second-most-common texture format in production glTF (Shopify AR, IKEA, Blender’s default export in 3.4+).

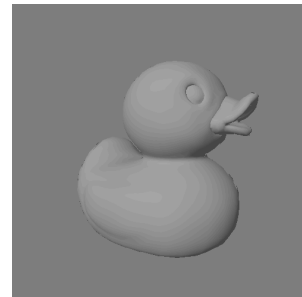
```
#render-gltf(duck-webp, read: R,  
  camera: (2.5, 1.5, 2.5), center: (0, 0, 0), up: (0, 1, 0),  
  fov: 40,  
  width: 40%,  
)
```



Unsupported formats

KHR_texture_basisu (KTX2 / Basis Universal) and EXT_texture_avif aren’t decoded — they’d pull in 1 MB of decoder deps for formats we ship without today. Assets using them render with a **placeholder white texture** per material (geometry stays intact) rather than crashing. Below: the same Duck with KTX2 textures, silhouette-shaded because the base color reads as white:

```
#render-gltf(duck-ktx, read: R,  
  camera: (2.5, 1.5, 2.5), center: (0, 0, 0), up: (0, 1, 0),  
  fov: 40,  
  background: "#333",  
  width: 40%,  
)
```



Workaround: pre-transcode via [gltf-transform](https://gltf-transform.dev) to WebP or PNG.

Metadata

`get-gltf-info` returns a dict — bounding box, triangle count, animation length. Useful for driving auto-framing or a scrub slider without touching the render path.

```
#let info = get-gltf-info(helmet, read: R)
#{
  raw(block: true, lang: "yaml",
    "triangles: " + str(info.triangles) + "\n" +
    "bbox_min: [" + info.bbox_min.map(x => str(calc.round(x, digits: 2))).join(", ")
+ "]" + "\n" +
    "bbox_max: [" + info.bbox_max.map(x => str(calc.round(x, digits: 2))).join(", ")
+ "]" + "\n" +
    "center: [" + info.center.map(x => str(calc.round(x, digits: 2))).join(", ") +
+ "]" + "\n" +
    "radius: " + str(calc.round(info.radius, digits: 2)) + "\n" +
    "max_animation_time: " + str(info.max_animation_time)
  )
}
```

```
triangles: 15452
bbox_min:  [-0.95, -0.9, -1.19]
bbox_max:  [0.94, 0.9, 0.81]
center:    [0, 0, -0.19]
radius:    1.64
max_animation_time: 0
```


What's supported vs. not

Core spec: glTF 2.0 JSON + GLB (single-file and split with external .bin/textures), meshes (POINTS/LINES/TRIANGLES), skinning (JOINTS_0 + JOINTS_1, WEIGHTS_0 + WEIGHTS_1, up to 8 influences per vertex), morph targets, animations (TRS + morph weights, all three interpolation modes), multiple UV sets (TEXCOORD_0/1/2), vertex colors, cameras (perspective + orthographic), multiple scenes, multiple animations, textures with samplers, sparse accessors, KHR_mesh_quantization.

Rendering: Cook-Torrance GGX PBR, IBL from HDR (Radiance / RGBE), shadow maps (PCF + PCSS), alpha modes (OPAQUE / MASK / BLEND via WBOIT), double-sided normal flip, SSAO, FXAA, SSAA $\times 2/\times 4$.

Extensions: KHR_lights_punctual, KHR_materials_unlit, KHR_materials_transmission, KHR_materials_ior, KHR_materials_specular, KHR_materials_emissive_strength, KHR_materials_volume, KHR_materials_clearcoat, KHR_materials_sheen, KHR_materials_iridescence, KHR_materials_anisotropy, KHR_materials_dispersion, KHR_materials_diffuse_transmission, KHR_texture_transform, KHR_materials_pbrSpecularGlossiness, KHR_materials_variants, EXT_texture_webp, EXT_meshopt_compression, KHR_draco_mesh_compression.

Not supported (assets render with graceful fallback — placeholder white texture or silent skip):

- KHR_texture_basisu (KTX2). Biggest gap — dominant production texture format.
- EXT_texture_avif. Rare in the wild.
- KHR_animation_pointer. Animations targeting material / light / camera properties via JSON pointer are ignored.
- TEXCOORD_N for $N \geq 3$ (collapses to slot 2).
- Sparse accessors combined with quantization.

See [crates/maquette-gltf/README.md](#) for the full compliance matrix and per-extension notes.